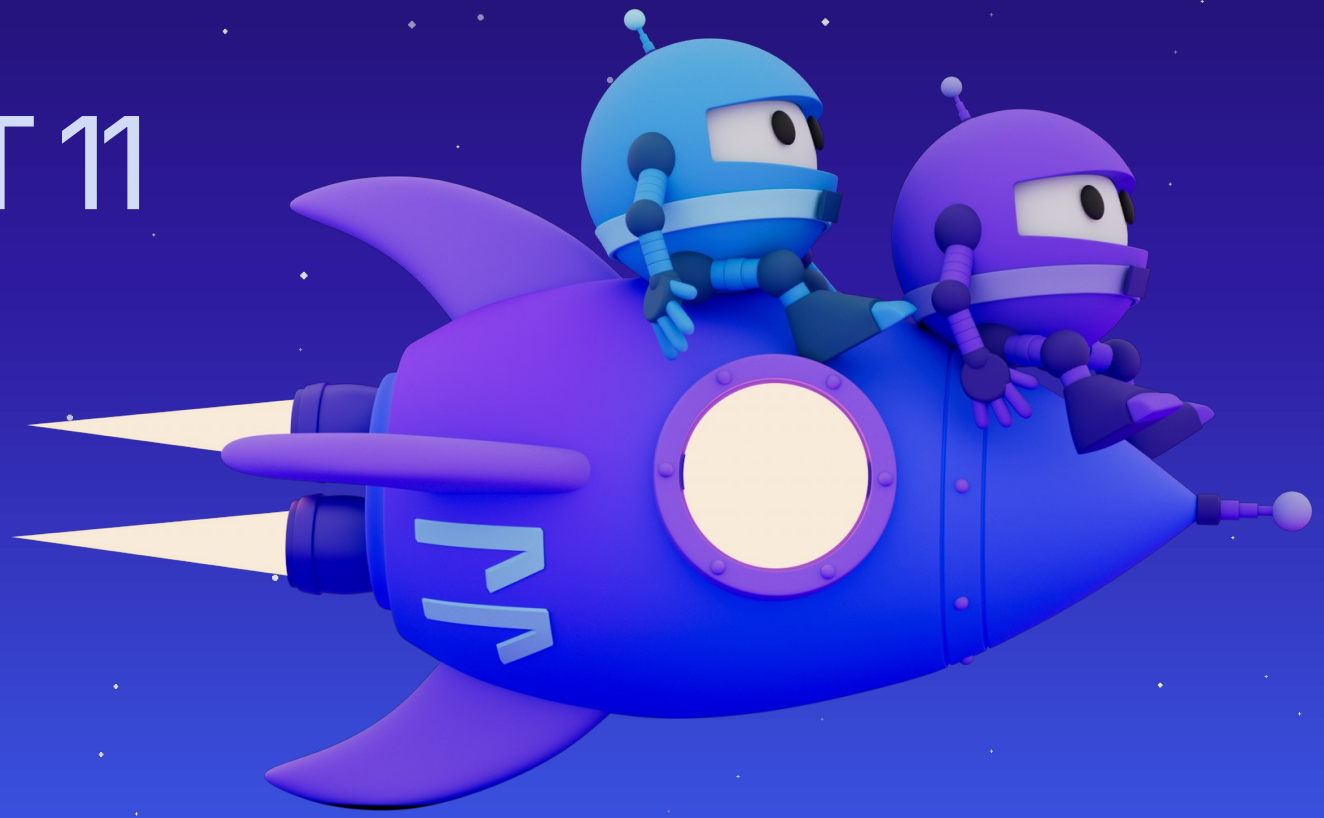
Infrastructure as (Actual) Code

with C# 15 and .NET 11

.NET



Cam Soper

Systems engineering, Pulumi

Infrastructure as code, they said

main.tf

```
variable "env" { type = string }

resource "azurerm_storage_account" "site" {
  name                     = "site${var.env}"
  resource_group_name      = "rg-${var.env}"
  location                 = "centralus"
  account_tier             = "Standard"
  account_replication_type = var.env == "prod" ? "ZRS" : "LRS"
}
```

Infrastructure as code, they said

main.tf

```
resource "azurerm_storage_account" "site" {
  # name, resource group, location, tier, replication: as before
}

resource "azurerm_management_lock" "keep" {
  count      = var.env == "prod" ? 1 : 0
  name       = "do-not-delete"
  scope      = azurerm_storage_account.site.id
  lock_level = "CanNotDelete"
}
```

Infrastructure as code, they said

main.tf

```
resource "azurerm_storage_account" "site" {
  # name, resource group, location, tier, replication: as before

  dynamic "blob_properties" {
    for_each = var.env == "prod" ? [1] : []
    content {
      versioning_enabled = true
    }
  }
}

resource "azurerm_management_lock" "keep" {
  # count, name, scope, lock level: as before
}
```

That's not code.
That's config with
extra steps.



Cam Soper

Systems engineering, Pulumi

13 years at Microsoft: .NET and Azure, co-host of On .NET Live

At Pulumi: built our automated docs QA system. Now: our agentic SDLC

What “actual code” gets you

Things C# devs already have

A compiler

Policy.cs

```
Redundancy(Stage stage) => stage switch
✖ error CS8509: ...the pattern
  'FridayAfternoon' is not covered.
```

Types and IntelliSense

Program.cs

```
SkuName.Standard_
```

- Standard_LRS
- Standard_RAGRS
- Standard_ZRS

Zone-redundant storage: copies across three availability zones.

Tests

bash

```
$ dotnet test
✔ Test run summary: Passed!
  succeeded: 7   failed: 0
```

Packages

bash

```
$ dotnet add package Pulumi.AzureNative
✔ PackageReference for 'Pulumi.AzureNative'
  version '3.30.0' added
```

Pulumi, in one slide

Program.cs

```
var sku = env == "prod" ? SkuName.Standard_ZRS : SkuName.Standard_LRS;
var storage = new StorageAccount("site", new()
{
    // name, resource group, location: as before
    Sku = new SkuArgs { Name = sku },
});

if (env == "prod")
{
    new ManagementLockByScope("keep", new()
        { Scope = storage.Id, Level = LockLevel.CanNotDelete });
    new BlobServiceProperties("versioning", new()
        { IsVersioningEnabled = true /* + which account */ });
}
```

Also: TypeScript · JavaScript · Python · Go · F# · VB · Java · YAML · **HCL**

01 Demo: Hello, cloud



C# 15: union types

Exactly one of a fixed set

Miss a case: compiler error

No default arm needed

Stages.cs

```
public record Dev;
public record Staging;
public record Prod(string OnCall);

public union Stage(Dev, Staging, Prod);

SkuName Redundancy(Stage s) => s switch
{
    Dev     => SkuName.Standard_LRS,
    Staging => SkuName.Standard_LRS,
    Prod    => SkuName.Standard_ZRS,
};
```

02 Demo: The compiler is your policy engine



A rule for my agents. In bold.

From my own CLAUDE.md:

“Do NOT put Access in front of hades: it self-implements OAuth and would double-prompt.”

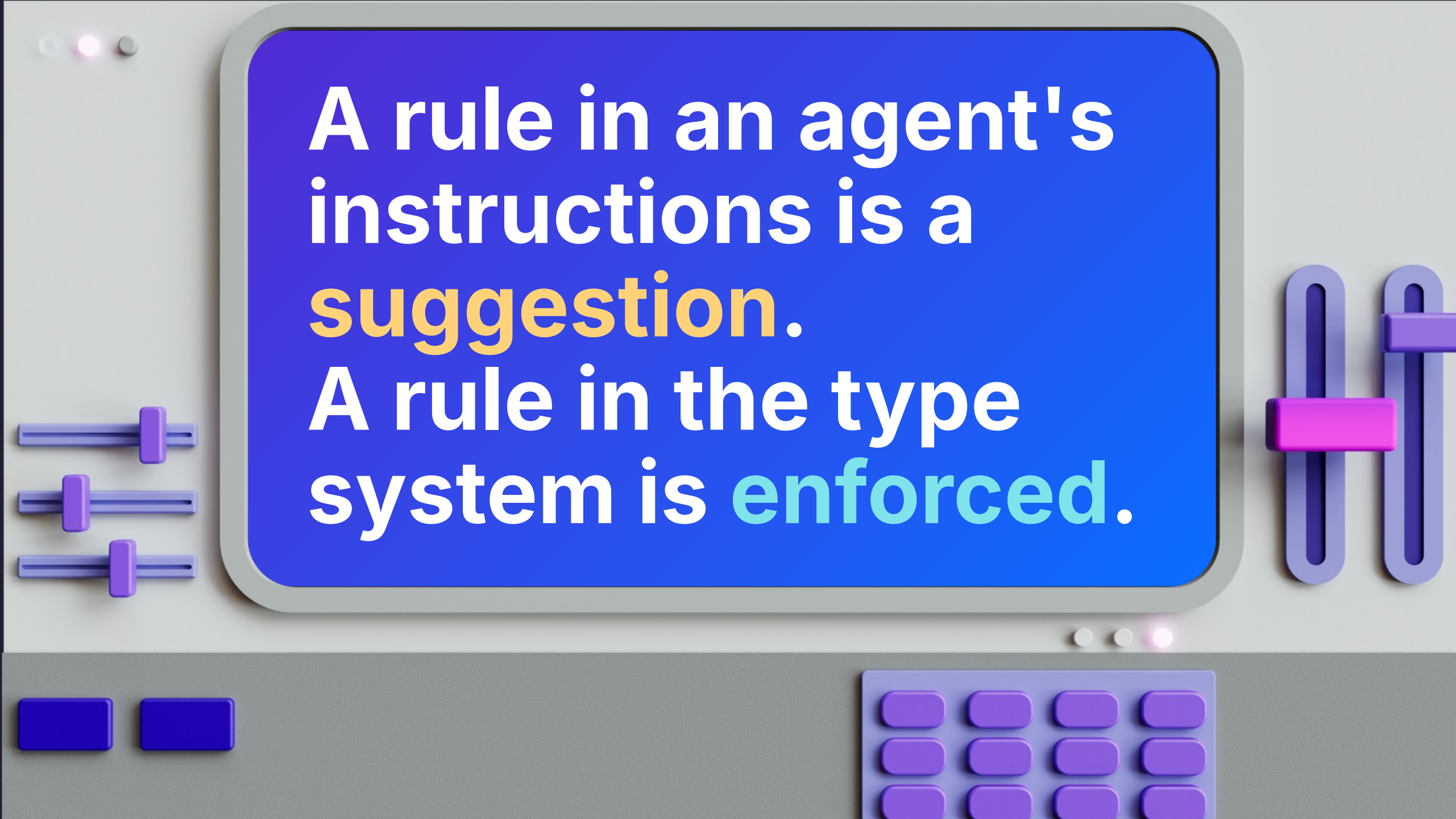
Written by Claude.
Checked in by me. Unread.

McpAuth.cs

```
public union McpAuth(
    SelfOAuth,
    AccessEdgeOnly,
    AccessWithOriginCheck);

record McpService(
    string Name, McpAuth Auth);

var hades = new McpService(
    "hades", new SelfOAuth());
// One service. One posture.
```


A rule in an agent's instructions is a suggestion.
A rule in the type system is enforced.

Take it home
Slides, code, and notes

`demos/`

Both demos, plus a bonus: agent vs. compiler

`tests/`

The whole stack vs. a fake cloud. No Azure.

`bonus/single-file/deploy.cs`

One file. dotnet run.

Gotcha

```
#:property PublishAot=false
```



github.com/CamSoper/infrastructure-as-actual-code

Thank you

github.com/CamSoper/infrastructure-as-actual-code

pulumi.com/docs/iac/languages-sdks/dotnet

“What's new in C# 15” on Microsoft Learn

